VertexMind SQL Internship

Step-by-Step Query Submission with Results

Dataset: **employees** and **departments** tables (sample company database). If your dashboard provides a different schema, swap table/column names below — the query logic and structure remain the same.

employees(employee_id, first_name, last_name, department_id, salary, hire_date)
departments(department_id, department_name, location)

1. Basics

Select all employees ordered by salary (highest first)

```
SELECT employee_id, first_name, last_name, salary
FROM employees
ORDER BY salary DESC;
```

employee_id	first_name	last_name	salary
108	Ananya	Das	102000.0
101	Arjun	Mehta	95000.0
102	Priya	Sharma	88000.0
109	Rohit	Kapoor	75000.0
105	Karthik	Raj	71000.0
106	Divya	Nair	67000.0
103	Rahul	Verma	62000.0
104	Sneha	Iyer	58000.0
107	Vikram	Singh	54000.0
110	Meera	Pillai	49000.0

10 row(s) returned

Select employees hired after 2021-01-01

```
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date > '2021-01-01'
ORDER BY hire_date;
```

first_name	last_name	hire_date
Arjun	Mehta	2021-03-15
Divya	Nair	2021-09-09
Rahul	Verma	2022-01-10
Sneha	Iyer	2022-05-20
Vikram	Singh	2023-02-14

first_name	last_name	hire_date
Meera	Pillai	2023-08-30

6 row(s) returned

2. Aggregate

Count employees and average salary per department

```
SELECT department_id, COUNT(*) AS num_employees, ROUND(AVG(salary),2) AS avg_salary
FROM employees
WHERE department_id IS NOT NULL
GROUP BY department_id;
```

department_id	num_employees	avg_salary
1	3	95000.0
2	2	60000.0
3	2	69000.0
4	1	54000.0
5	1	75000.0

5 row(s) returned

Departments where total salary exceeds 100000

```
SELECT department_id, SUM(salary) AS total_salary
FROM employees
WHERE department_id IS NOT NULL
GROUP BY department_id
HAVING SUM(salary) > 100000;
```

department_id	total_salary
1	285000.0
2	120000.0
3	138000.0

3 row(s) returned

3. Joins

Inner join: employees with their department name

```
SELECT e.first_name, e.last_name, d.department_name, d.location
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id;
```

first_name	last_name	department_name	location
Arjun	Mehta	Engineering	Bangalore

first_name	last_name	department_name	location
Priya	Sharma	Engineering	Bangalore
Rahul	Verma	Sales	Mumbai
Sneha	Iyer	Sales	Mumbai
Karthik	Raj	Marketing	Delhi
Divya	Nair	Marketing	Delhi
Vikram	Singh	HR	Chennai
Ananya	Das	Engineering	Bangalore
Rohit	Kapoor	Finance	Hyderabad

9 row(s) returned

Left join: all employees, department name if available

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id;
```

first_name	last_name	department_name
Arjun	Mehta	Engineering
Priya	Sharma	Engineering
Rahul	Verma	Sales
Sneha	Iyer	Sales
Karthik	Raj	Marketing
Divya	Nair	Marketing
Vikram	Singh	HR
Ananya	Das	Engineering
Rohit	Kapoor	Finance
Meera	Pillai	

10 row(s) returned

4. Complex

Subquery: employees earning above the company average salary

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

first_name	last_name	salary
Ananya	Das	102000.0
Arjun	Mehta	95000.0

first_name	last_name	salary
Priya	Sharma	88000.0
Rohit	Kapoor	75000.0

4 row(s) returned

CASE statement: classify employees into salary bands

```
SELECT first_name, last_name, salary,
  CASE
    WHEN salary >= 90000 THEN 'High'
    WHEN salary >= 65000 THEN 'Medium'
    ELSE 'Low'
  END AS salary_band
FROM employees
ORDER BY salary DESC;
```

first_name	last_name	salary	salary_band
Ananya	Das	102000.0	High
Arjun	Mehta	95000.0	High
Priya	Sharma	88000.0	Medium
Rohit	Kapoor	75000.0	Medium
Karthik	Raj	71000.0	Medium
Divya	Nair	67000.0	Medium
Rahul	Verma	62000.0	Low
Sneha	Iyer	58000.0	Low
Vikram	Singh	54000.0	Low
Meera	Pillai	49000.0	Low

10 row(s) returned

5. Submit

Final summary: department-wise employee count and avg salary, joined with department names

```
SELECT d.department_name, d.location, COUNT(e.employee_id) AS num_employees,
       ROUND(AVG(e.salary),2) AS avg_salary
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id
ORDER BY avg_salary DESC;
```

department_name	location	num_employees	avg_salary
Engineering	Bangalore	3	95000.0
Finance	Hyderabad	1	75000.0
Marketing	Delhi	2	69000.0

department_name	location	num_employees	avg_salary
Sales	Mumbai	2	60000.0
HR	Chennai	1	54000.0

5 row(s) returned